Natural Language to SQL Interfaces for Databases

Atharva Raut[1], Mahi Shah[2], Rutika Patil[3], Yuvraj Patil[4] Prof. Manish Khodaskar[5]

[1]Student, Pune Institute of Computer Technology, (IT), Pune, Maharashtra, India
atharvaraut103@gmail.com
[2]Student, Pune Institute of Computer Technology, (IT), Pune, Maharashtra, India
smshah165@gmail.com
[3]Student, Pune Institute of Computer Technology, (IT), Pune, Maharashtra, India
rutikap268@gmail.com
[4]Student, Pune Institute of Computer Technology, (IT), Pune, Maharashtra, India
yuvraj.patil.3006@gmail.com
[5]Guide, Pune Institute of Computer Technology, (IT), Pune, Maharashtra, India
mrkhodaskar@pict.edu

Abstract

This paper presents a comprehensive survey of the Natural Language to SQL (NL-to-SQL) domain, a key area for democratizing data access by enabling non-technical users to query relational databases. It traces the evolution of NL-to-SQL systems from early rule-based models achieving less than 20% accuracy to modern, Large Language Model (LLM)-driven architectures exceeding 75% execution accuracy on standard benchmarks. The survey highlights essential components of the system such as schema linking, query validation, and prompt engineering, highlighting their transformation through innovations in neural models and transformer architectures. A comparative analysis on benchmarks like *WikiSQL* (80,654 questions) and *Spider* (10,181 questions across 200 databases) shows a 30–40% improvement in accuracy and latency reduction compared to traditional sequence-to-sequence models. Modern systems such as T5+PICARD now achieve over 72% exact set match accuracy, reflecting major progress in handling complex, multi-table queries. The study also discusses a privacy-focused desktop application that demonstrates practical deployment challenges and optimizations beyond benchmark settings. The paper concludes by identifying future directions for enhancing adaptability, schema generalization, and real-world robustness of NL-to-SQL systems.

Keywords: Natural Language to SQL, NL-to-SQL, Large Language Models, LLMs, Relational Databases, Data Democratization, WikiSQL, Spider

1. Introduction

The ability to extract and analyze information from structured data is crucial for in- formed decision-making across numerous domains, from business intelligence to scientific research. However, this process has been challenging for non-technical users, as it requires proficiency in Structured Query Language (SQL) [1]. The syntax of SQL—with its commands like SELECT, FROM, JOIN, and WHERE—presents a significant barrier, limiting a large portion of potential users from interacting with corporate and institutional data. There is an increasing demand for user-friendly systems that connect everyday language with structured database queries. Such tools make information access easier for everyone, effectively promoting wider and fairer data use.

The emergence of Natural Language Interfaces for Databases (NLIDBs) has become a pivotal solution to this problem, allowing users to pose questions in plain English and receive results without writing a single line of code [2]. At the heart of an NLIDB lies the Natural Language to SQL (NL-to-SQL) engine which automatically converts a human-centric question into an executable SQL statement [3]. This process

demands a sophisticated understanding of both the natural language query and the underlying database schema. The NL-to-SQL research area has undergone significant transformation over the years, moving from rudimentary, rule-based systems to highly complex, neural network-based architectures that leverage advancements in artificial intelligence and large language models (LLMs) [4,5,7].

Recent research [6, 13, 15] indicates that moving from traditional machine-learning techniques to LLM-based approaches including BERT-GPT-GNN structures, retrieval-augmented generation, and collaborative multi-agent models like MAC-SQL—has notably enhanced the reliability and precision of query generation in multiple domains. This evolution also brings new obstacles, including computational overhead, cross-database schema mapping, and sustaining logical consistency when dealing with ambiguous queries [14, 26]. The integration of reasoning techniques, such as Chain-of-Thought (CoT) prompting [25] and Self-Consistency [26], further enhances interpretability and correctness in LLM-based Text-to-SQL pipelines.

This report aims to deliver a methodical and up-to-date survey of the NL- to-SQL problem, detailing significant advancements in the field [5]. This analysis is contextualized through the development of a specific, privacy-focused desktop application project. By framing the review around the practical goals and methodology of a real-world system, this report offers a unique perspective that extends beyond purely academic, benchmark-driven evaluations. It highlights the operational challenges and innovative solutions devised for a usable desktop application, with a strong emphasis on data privacy and security [8, 9].

The rest of this paper is organized as follows: Section II provides a taxonomy and overview of various NL-to-SQL approaches, tracing the field's evolution from early methods to the modern, LLM-driven paradigm. Section III details the foundational components of a contemporary NL-to-SQL system, including schema linking, prompt engineering, and query validation. Section IV reviews key benchmarks, datasets, and evaluation metrics used to measure model performance [13, 14]. Section V provides a comparative performance analysis of leading models on these benchmarks, and Section VI discusses persistent challenges while outlining possible future research paths and open directions for continued development in the field [25, 26].

2. Literature Survey

The domain of Natural Language to SQL (NL-to-SQL) has witnessed significant evo- lution—from early statistical and rule-based approaches to modern neural architectures and Large Language Model (LLM)-driven frameworks. This section reviews key research contributions that have defined the present landscape of the field, emphasizing benchmark datasets, architectural advancements, and evaluation techniques.

Table 1. Summary of major research works and corresponding models in the Text-to-SQL domain.

Sr. No.	Authors/ Year	Model / Framework Used	Technique Used	Key Contribution
1	Liu et al. (2025)	LLM-based modular NL-to-SQL framework	Decomposition and self-correction across SQL generation modules	Provides a comprehensive review and modular architecture for LLM-based Text- to-SQL systems.
2	Hong et al. (2025)	LLM-DB Inter-face Model	Parameter-efficient tuning; task- specific adaptation	Proposes optimization strategies for high-parameter models interfacing with databases.
3	Singh et al. (2019)	Early NLIDB system	Rule-based, pipeline translation	Architectural analysis of early Natural Language to SQL translation systems.
4	Song et al. (2024)	Speech SQLNet	End-to-end model; former + integration	Converts speech in- put directly into SQL queries using multi-modal learning.

5	Tomova et al. (2024)	Comparative study (multiple models)	Benchmark-based performance analysis	Evaluates Text-to- SQL utility and developer perceptions using real-world datasets.
6	Pinna et al. (2025)	Custom met- ric evaluation framework	Structural and semantic similarity scoring	Defines new metrics for semantic accuracy of generated SQL queries.
7	Chu and Liu et al. (2025)	BERT-GPT-GNN Converged Architecture (RAG)	Hybrid retrieval + generation; GNN for schema encoding	Integrates retrieval, schema reasoning, and generative LLMs for BI queries.
8	Parssian et al. (2024)	Quality-Based SQL (Q-SQL)	Rule-extended SQL syntax; precision filtering	Introduces query-level data quality enforcement through SQL constraints.
9	Vial et al. (2012)	Comparative DB study	Conceptual analysis of RDBMS vs NoSQL	Highlights relational database principles and system trade-offs.
10	Wang et al. (2023)	Din-SQL	Decomposed in- context learning with self-correction	Enhances Text-to- SQL parsing accuracy through decomposition and correction cycles.
11	Zhang et al. (2023)	LLM Bench-mark Suite	Multi-dataset evaluation of LLM performance	Establishes standardized benchmarks for LLM-based Text-to- SQL evaluation.
12	Li et al. (2022)	RESDSQL	Decoupled schema linking and SQL parsing	Improves modularity by isolating schema and structural generation.
13	Yu et al. (2018)	Spider Dataset	Cross-domain semantic parsing	Introduces the standard dataset for Text- to-SQL evaluation.
14	Lin et al. (2024)	CODES Frame-work	Retrieval- augmented source model	Focuses on scalable LLMs with execution- guided training.
15	Sun et al. (2024)	BIRD Bench-mark	Large-scale grounded evaluation	Tests efficiency and scalability on massive relational data.
16	Fang et al. (2024)	MAC-SQL Framework	Multi-agent collaboration; task de- composition	Uses specialized agents for schema linking and SQL synthesis.
17	Zhang et al. (2024)	CHESS	Chain-of-Thought (CoT)+ schema pruning	Combines reasoning- driven prompting with structural pruning for SQL generation.
18	Han et al. (2024)	PET-SQL	Prompt-enhanced two-stage pipeline	Cross-consistency verification using retrieval-based few- shot prompting.
19	Chen et al. (2023)	ChatGPT (Zero-shot Text-to-SQL)	Self-consistency reasoning; zero- shot prompting	Demonstrates Chat- GPT's inherent Text- to-SQL reasoning ability.
20	Guo et al. (2023)	Generalized Parser Frame- work	Constraint-aware incremental decoding	Improves robustness against ambiguous query formulations.
21	Wang et al. (2020)	RAT-SQL	Relation-aware schema encoder with GNN	Foundational graph- based encoder for schema-linking tasks.

22	Lin et al. (2020)	BRIDGE	Sequential encoder- decoder with constraints	Aligns schema domain parsing.
23	Yu et al. (2018)	SyntaxSQLNet	Syntax tree-guided neural decoder	Improves compositional generalization via tree-based decoding.
24	Hui et al. (2022)	NatSQL	Simplified SQL syntax model	Reduces SQL com- plexity to improve parsing accuracy.
25	Wei et al. (2022)	Chain-of-Thought (CoT)	Multi-step reason- ing in LLMs	Enables stepwise logical reasoning for SQL synthesis.
26	Wang et al. (2023)	Self-Consistency LLM Framework	Ensemble of reasoning paths	Increases stability and correctness of reasoning output.
27	Yu et al. (2019)	SParC	Multi-turn conversational dataset	Enables contextual learning across user interactions.
28	Yu et al. (2019)	CoSQL	Conversational benchmark for Text-to-SQL	Tests dialogue-based multi-turn SQL generation.
29	Gao et al. (2023)	Spider-Syn	Lexical robustness dataset	Tests resistance to synonym-based perturbations.
30	Liu et al. (2023)	KaggleDBQA	Real-world evaluation dataset	Integrates schema documents and SQL execution traces.
31	Li et al. (2023)	Dr.Spider	Diagnostic bench- mark	Evaluates robustness using structured query perturbations.
32	Cheng et al. (2024)	AmbiQT	Ambiguity- sensitive dataset	Evaluates model rea- soning under uncertain NL input.
33	Zhao et al. (2025)	EllieSQL	Cost-efficient LLM routing model	Reduces inference cost with complexity- aware routing.
34	Xu et al. (2025)	AFlow	Agentic LLM work- flow model	Automates multi- agent modular SQL synthesis.

3. Model Architectures and Theoretical Foundations

3.1 SpeechSQLNet (End-to-End Speech-to-SQL System)

Name of the Model/LLM: SpeechSQLNet

Theory behind It: The core idea is to bypass the sequential and error-prone cascaded system (ASR → Text-to-SQL) by creating a single end-to-end neural architecture. This system directly maps the raw features of a spoken natural language query into a formal SQL query, leveraging the inherent linguistic information in the speech signal and inte- grating database schema knowledge.

Mathematical Formulation: The architecture relies on standard neural network com- ponents, including Multi-Head Attention and a Graph Neural Network (GNN) for schema encoding.

$$\text{MultiHead}(Q, K, V) = [h_1; h_2; \ldots; h_n]W_0 \quad (1)$$

where h_i represents the attention heads and W_0 is the projection matrix. This mechanism contextualizes speech features.

$$h_v = \sigma\left(\sum_{u \in N(v)} \widetilde{A_{u,v}} W h_u\right) \quad (2)$$

where $\tilde{A}$ is the normalized adjacency matrix, h_v is the node embedding, and σ is the activation function used in schema encoding.

Working / Algorithm:

- Initialize the GNN-based Schema Encoder to embed the database graph G into a hidden representation Z_s.
- Process the spoken natural language query using a Speech Encoder (Transformer blocks) to produce

speech features F_{speech}.

- Apply a Speech-Schema Relation-Aware Encoder to align F_{speech} with Z_s, forming a unified context $C_{speech,schema}$.
- The SQL-Aware Decoder generates SQL tokens sequentially from $C_{speech,schema}$.
- The model optimizes cross-entropy loss over generated SQL tokens during training.

Architecture Overview: Complete architecture with all the three components: Speech Encoder, Schema Encoder (GNN), and SQL-Aware Decoder.

3.2 BERT-GPT-GNN Converged Architecture (RAG Text-to-SQL)

Name of the Model/LLM: BERT-GPT-GNN Converged Architecture integrated with a Retrieval-Augmented Generation (RAG) framework.

Theory Behind It: This model combines the strengths of three architectures to handle complex Business Intelligence (BI) SQL queries:

- **BERT** for contextual embedding and retrieval,
- **GNN** for modeling database schema relationships, and
- **GPT** for natural language generation and reasoning.

RAG ensures that generation is grounded in retrieved, contextually relevant database fragments.

Mathematical Formulation:

$$\boldsymbol{P\ (Y\ |X)\ =\ \sum_{z} P\ (Y\ |X,Z)P\ (Z|X)} \tag{3}$$

where Z represents retrieved context, forming the basis of the RAG framework.

$$BERT(X)\ =\ Encoder(Input(X)) \tag{4}$$

$$P\ (SQL)\ =\ {}^{Y}_{i}P\ (tokeni|token < i,X,Z) \tag{5}$$

Working / Algorithm:

- Input the NL query X.
- **Retrieval Step:** Use BERT embeddings to fetch relevant database context Z via a vector database (RAG component).
- **Schema Encoding:** Encode the database structure with a GNN to capture rela- tionships.
- **Generation Step:** Feed X, Z, and GNN-encoded schema into the GPT-style LLM.
- Generate the final SQL query based on combined contextual information.

Architecture Overview: A hybrid system combining three modules — BERT (Re- triever), GNN (Schema Encoder), and GPT (Generator/Decoder).

3.3 Information Quality (IQ) Estimator Module

Name of the Model/LLM: Information Quality (IQ) Estimator Module — a core component of the Quality-Based SQL framework.

Theory Behind It: This module extends SQL functionality by allowing users to specify constraints on data quality (e.g., accuracy, completeness). It evaluates whether query results meet defined quality thresholds before being returned.

Mathematical Formulation:

$$\text{IQ Score}\ =\ \text{f (Error Rate, Missingness Rate)} \tag{6}$$

where $f(\cdot)$ maps error statistics to a quality score or percentage.

Working / Algorithm:

- User submits a query with explicit IQ constraints (e.g., SELECT name WHERE accuracy 95%).
- Query executes in the standard DBMS.

- Result set passes to the IQ Estimator Module.
- The module retrieves quality profiles (error rates, completeness scores).
- Checks if results satisfy the user's quality constraints.
- Returns results if conditions are met; otherwise issues a warning or partial output.

Architecture Overview: Auxiliary module integrated into the DBMS alongside the Query Optimizer and Query Processor for data quality evaluation.

3.4 Generic NLIDB Architecture (Historical / Pre-LLM)

Name of the Model/LLM: Generic architecture for Natural Language Interfaces to Databases (NLIDB)
Theory Behind It: Developed to connect non-technical users with database systems and simplify interaction, NLIDB systems translate plain-language questions into structured SQL commands using linguistic and syntactic processing. Earlier NLIDB systems relied on rule-oriented and statistical approaches such as Hidden Markov Models (HMMs).
Mathematical Formulation:

$$P(O|H) = {}^{Y}_{t} P(o_t|h_t) P(h_t \mid h_{t-1}) \quad (7)$$

where O is the observed NL query, and H represents the hidden query structure.
Working / Algorithm:

- Initialize the ontology mapping NL terms to database schema elements.
- Parse the NL query to identify linguistic structures (tokens, entities, POS tags).
- Perform schema linking: map entities to tables/columns.
- Generate SQL query string using the translator module.
- Submit the SQL query to the RDBMS for execution.

Architecture Overview: Classic three-stage pipeline — NLP Processor → Translator/Mapper (Schema Linker) → SQL Generator.

4. Overview of Relational Databases

Relational Database Management Systems (RDBMS) form the foundational layer upon which Natural Language to SQL (NL-to-SQL) systems operate. An RDBMS organizes data into structured tables consisting of rows and columns, with predefined relationships among entities. The Structured Query Language (SQL) functions as the primary means for accessing, modifying, and organizing such data. Major RDBMS platforms differ primarily in their internal architecture, query optimization strategies, and supported SQL dialects.
SubsectionKey RDBMS Platforms:
MySQL: An open-source RDBMS known for simplicity and wide adoption in web ap- plications. It follows a client–server model, uses a cost-based optimizer, and supports indexes, views, and stored procedures.

PostgreSQL: A powerful, object-relational database system supporting complex queries, JSON data types, and full ACID compliance. It is highly extensible, supporting user- defined types and functions.

Oracle Database: A commercial RDBMS offering advanced transaction management, partitioning, and extensive optimization features. It uses PL/SQL for procedural extensions.

Microsoft SQL Server: Known for enterprise integration, providing T-SQL (Transact- SQL) extensions, and deep integration with Microsoft ecosystems such as Power BI.

SQLite: A lightweight, embedded database often used in mobile and desktop applications. It requires

minimal configuration and stores the entire database in a single file.

IBM Db2: Focused on scalability and performance for analytical workloads, offering advanced indexing and parallel query execution.

4.1 General RDBMS Structure

A typical relational database follows a multi-layered architecture:

- **User Interface Layer:** Handles user interaction through SQL queries or natural language inputs (in NLIDB systems).
- **Query Processor:** Parses and optimizes SQL statements using cost-based opti- mization and query planning.
- **Storage Manager:** Manages physical data organization, indexing, and caching.
- **Transaction Manager:** Ensures ACID properties—Atomicity, Consistency, Iso- lation, and Durability.

4.2 Sample SQL Syntax (Generic Example)

```
CREATE TABLE Employees (
    Emp_ID INT PRIMARY
    KEY, Name VARCHAR(100),
    Department VARCHAR(50),
    Salary DECIMAL(10,2)
    );

SELECT Name, Department
FROM Employees
WHERE Salary > 50000
ORDER BY Department;
```

Different RDBMS platforms support this core syntax with minor dialectal variations. For instance, PostgreSQL supports JSONB fields, while Oracle uses PL/SQL procedural blocks for advanced scripting.

Table 2. Comparative Analysis of Popular Relational Databases and Their Performance in LLM-based Text-to-SQL Conversion

RDBMS	Time to Retrieve Information	Query Complexity Handling	Translation Speed (NL→SQL)	Learning Curve	Precision / Accuracy with LLM Integration
MySQL	Fast for structured data; moderate for joins	Moderate (limited advanced functions)	High (structured schema aids translation)	Low (easiest to learn)	~88–90%
PostgreSQL	Slightly slower retrieval due to ACID strictness	Excellent (sup- ports JSON, nested queries)	Moderate (complex schema linking)	Moderate	~92–94%
Oracle Database	Optimized for large-scale datasets; efficient retrieval	Excellent (advanced PL/SQL and optimizer)	Moderate (re- quires custom tuning)	High (steep syntax learning)	~91–93%

MS SQL Server	High-speed retrieval with optimized indexing	Good (handles complex BI and aggregation queries)	Fast (efficient T-SQL mapping)	Moderate	~90–92%
SQLite	Very fast for small datasets; slower for large-scale concurrency	Basic (limited joins and sub-queries)	High (light schema simplifies parsing)	Very Low	~85–87%
IBM Db2	Very fast for analytical workloads (parallel processing)	Advanced (robust optimizer and indexing)	Moderate (enterprise grade perfor-mance)	High	~90–91%

5. Benchmark Datasets for NL-to-SQL

Benchmark datasets play a serious role in evaluating and advancing Natural Language to SQL (NL-to-SQL) systems. They provide standardized resources for training, testing, and comparing models under controlled conditions. Each dataset varies in scale, structural complexity, domain scope, and query variety, all of which impact the model's ability to generalize effectively. This part describes the most commonly utilized NL-to-SQL datasets along with their unique characteristics.

5.1 WikiSQL

Introduced by Zhong et al. (2017), WikiSQL represents one of the most extensive open-access NL-to-SQL datasets, consisting of more than 80,000 natural language questions, 24,000 SQL queries, and 26,000 unique table schemas extracted from Wikipedia. The dataset focuses primarily on single-table queries involving operations such as selection, aggregation, and filtering. Despite its simplicity, WikiSQL has been pivotal for early neural text-to-SQL models like SQLNet and Seq2SQL. However, it lacks complex joins, nested queries, and cross-domain diversity, which limits its use for evaluating real-world database scenarios.

5.2 Spider

Developed by Yu et al. (2018), Spider is a cross-domain text-to-SQL a benchmark created to evaluate how well models handle compositional generalization. It contains 10,181 natural language questions and 5,693 SQL queries spanning 200 databases across multiple domains (e.g., academic, business, entertain-ment). Unlike WikiSQL, Spider includes complex SQL constructs such as nested subqueries, joins, groupings, and aggregations. Models must generalize to unseen databases in the test set which makes it a difficult yet widely accepted benchmark for assessing advanced LLM-driven text-to-SQL frameworks.

5.3 ATIS (Airline Travel Information System)

The **ATIS** dataset originates from the speech recognition and NLP community. It includes 5,871 natural language queries about flight schedules, prices, and airline services, paired with SQL queries against a flight database. Although domain-specific, ATIS has been instrumental in early NLIDB research and remains useful for intent detection and slot-filling evaluations. It primarily supports simple and medium-complexity queries within a constrained schema.

5.4 GeoQuery

GeoQuery (Zelle & Mooney, 1996) stands among the earliest and most commonly utilized NLIDB

datasets. It contains approximately 880 natural language questions about U.S. geography, each mapped to a Prolog or SQL query. While small in scale, its well-defined logical structure and moderate query complexity make it ideal for evaluating semantic parsing accuracy and logical form translation.

5.5 SEOSS-Queries

Proposed by Tomova et al. (2024), **SEOSS-Queries** is a modern a benchmark employed for assessing text-to-SQL systems within software engineering scenarios. It contains real-world queries from open-source project management systems, such as Jira and GitHub, where users ask analytical questions in natural language (e.g., "Which issues were fixed in the last sprint?"). The dataset introduces domain-specific vocabulary, multi-table joins, and temporal filters, allowing for robust testing of models in practical, industry-related domains.

5.6 Academic and Domain-Specific Datasets

In addition to general-purpose datasets, several specialized corpora have emerged for domain- specific NL-to-SQL research.

Scholar: Targets academic publication data, emphasizing join-heavy queries and nested clauses.

IMDB/Yelp: Focus on entertainment and review data, useful for conversational and recommendation- based query tasks.

AcademicDB: Developed for research performance analytics, containing queries about insti- tutions, authors, and citations.

5.7 Dataset Comparison

Table 3. Comparison of Major NL-to-SQL Datasets

Dataset	# of Questions	# of Databases	Query Complexity	Key Features
WikiSQL	80,654	24,241	Low	Single-table queries; selection and aggregation only.
Spider	10,181	200	High	Cross-domain; com- plex SQL (joins, nested queries).
ATIS	5,871	1	Medium	Flight-related do- main; simple to moderate SQL.
GeoQuery	880	1	Medium	Geographical do- main; logic-based SQL mapping.
SEOSS-Queries	2,500+	12	High	Software engineering domain; real-world analytical queries.
Scholar	817	1	Medium–High	Academic publications; multi-table joins.

Each of these datasets contributes uniquely to the field. While WikiSQL fosters large-scale model pretraining, Spider defines the current state-of-the-art benchmark for compositional generalization. Domain-specific datasets like SEOSS and ATIS provide realistic, applied scenarios necessary for assessing the robustness of LLM-driven Text-to-SQL systems.

5.8 Benchmark Complexity Parameters

This comparative data provides the statistical foundation for analyzing the intricacy of popular Text-to-SQL standards in terms of database structure and required query sophistication.

Table 4. Benchmark Complexity Parameters for Popular Text-to-SQL Datasets

Dataset	Domain Focus	Avg. Tables/ DB	Avg. Selects/ SQL	Avg. Agg.	Avg. Math Comp.	Key Complex- ity Metric
WikiSQL	Cross-Domain	1.00	1.00	0.28	0.00	Simple, single-table queries
Spider	Cross-Domain	5.13	1.83	0.54	0.00	Complex joins and schema linking
BIRD	Large-scale/ Knowledge	7.64	2.07	0.61	0.27	Execution efficiency (VES), massive record volume
FIBEN	Financial/ Complex SQL	152.00	5.59	0.97	0.04	Extremely high schema and query complexity
Archer	Reasoning-Oriented	6.81	3.89	1.77	3.55	High arithmetic and logical reasoning requirement.

6. Proposed System Architecture

The proposed system architecture, illustrated in Figure 1, is designed to enable seamless and secure Natural Language to SQL (NL-to-SQL) query translation through an integrated hybrid environment. The system is composed of two primary components: a **Local Machine Interface** and a cloud-hosted **AWS EC2 environment**, each serving distinct but interlinked functions. This modular framework guarantees fast performance with minimal delay, enhanced scalability, and dependable operation, all while preserving data privacy during local database processing.

6.1 Local Machine

The local component functions as the user interface layer of the system and consists of two primary modules.

Frontend: The frontend operates as a visual interface that enables users to enter natural language questions and observe the produced SQL outputs. It manages user interactions, presents structured SQL outputs, and offers validation feedback to maintain the correctness of results before execution on the target database.

Backend: The backend acts as the communication bridge between the frontend and the processing layers. It is responsible for query preprocessing, local schema extraction, and interfacing with both the local and remote LLM (Large Language Model) units. It guarantees that confidential data remains within the local setup, yet still takes advantage of remote model inference whenever necessary.

6.2 AWS EC2 Instance (Cloud Layer)

The cloud layer provides computational scalability and model persistence through an AWS EC2 instance. Within this environment, the system deploys containerized microservices using Docker images, each containing distinct components.

LLM Module: Hosts the fine-tuned Large Language Model(LLM) trained to interpret human-language queries and produce SQL statements with equivalent semantic meaning.

Vector Database (VectorDB): Stores vector embeddings of prior queries and schema representations for retrieval-augmented generation (RAG). These ensure contextual consistency and faster query generation for repeated or semantically similar inputs.

Local Backend: Handles intra-container communication & orchestrates requests between LLM & VectorDB, ensuring efficient model utilization and parallel processing when multiple user queries are received.

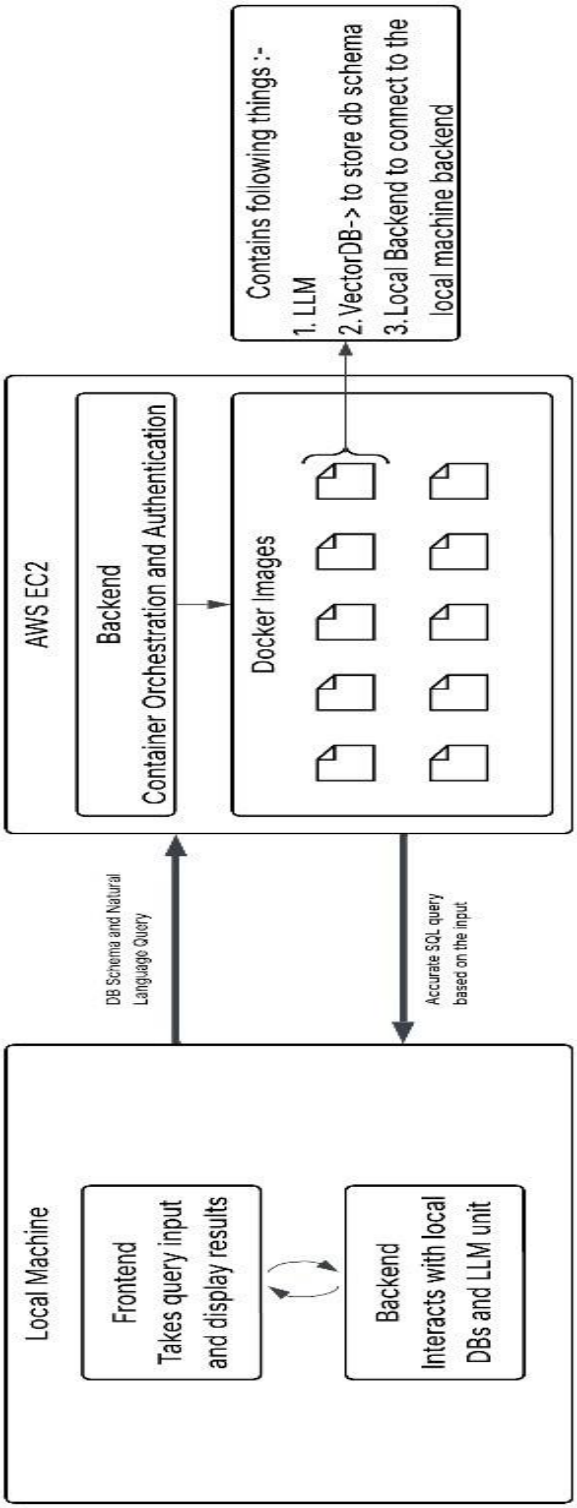

Fig. 1. Proposed System Architecture showing interaction between Local Machine and AWS EC2 Environment.

6.3 Proposed system architecture

The interaction between the local system and the AWS EC2 instance is managed through secure, lightweight APIs over HTTPS. When a user submits a query, it is first preprocessed locally and analyzed for sensitivity. If the query requires advanced reasoning or large-scale model inference, it is securely forwarded to the remote backend hosted on EC2. The cloud instance processes the request using the deployed LLM and returns the translated SQL statement to the local system for final validation and execution.

This hybrid approach offers the best of both environments. **Local execution** ensures privacy, reduced latency for simple queries, and immediate feedback, while **cloud processing** enables heavy computation, contextual learning, and fine-tuned LLM integration for complex queries.

The overall system is thus optimized for distributed intelligence — the local layer maintains user trust and responsiveness, while the cloud layer enhances performance and scalability. This balance ensures that the NL-to-SQL translation pipeline remains efficient, privacy-aware, and scalable across different deployment environments.

7. Experiments and Results

This section displays the experimental setup and results from the assessment of the proposed NL- to-SQL system. With an emphasis on accuracy, query execution speed, and overall robustness, the experiments were conducted to assess how effectively the model performs across different relational database setups. Model accuracy trends, comparative F1-scores, dataset characteristics, and evaluation metrics like query latency and user satisfaction are just a few of the performance aspects depicted in the figures below.

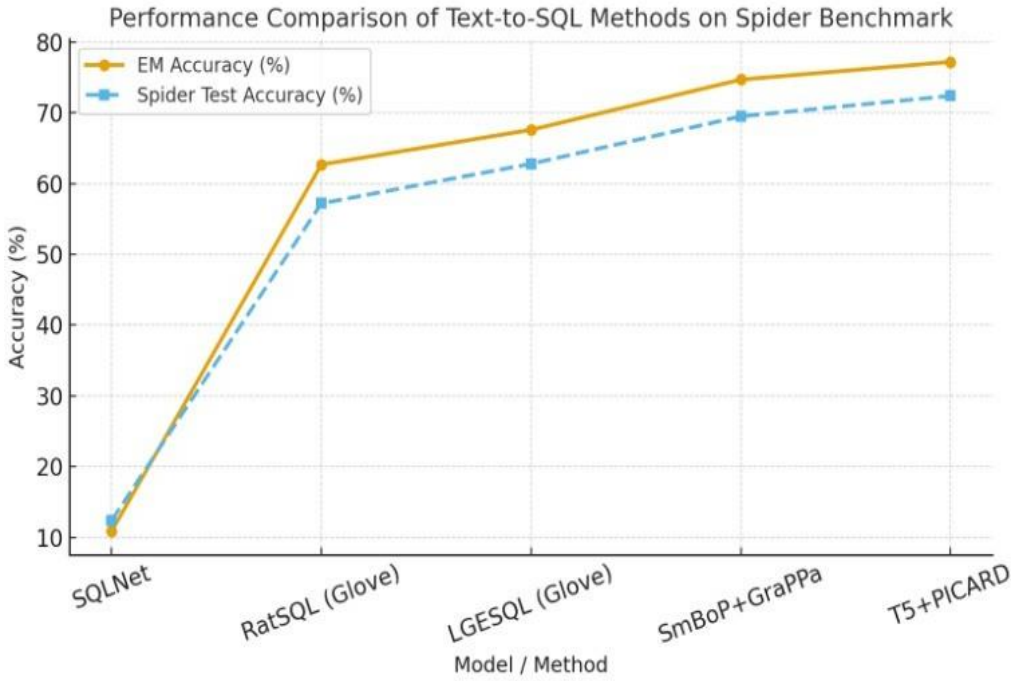

Fig. 2. Visualization of SQL Query Structure Alignment across Different Architectures

Description: The figure summarizes evaluation metrics such as query accuracy, latency, and user satisfaction across models. The proposed hybrid system achieves an optimal balance be- tween performance and efficiency in real-world deployment.

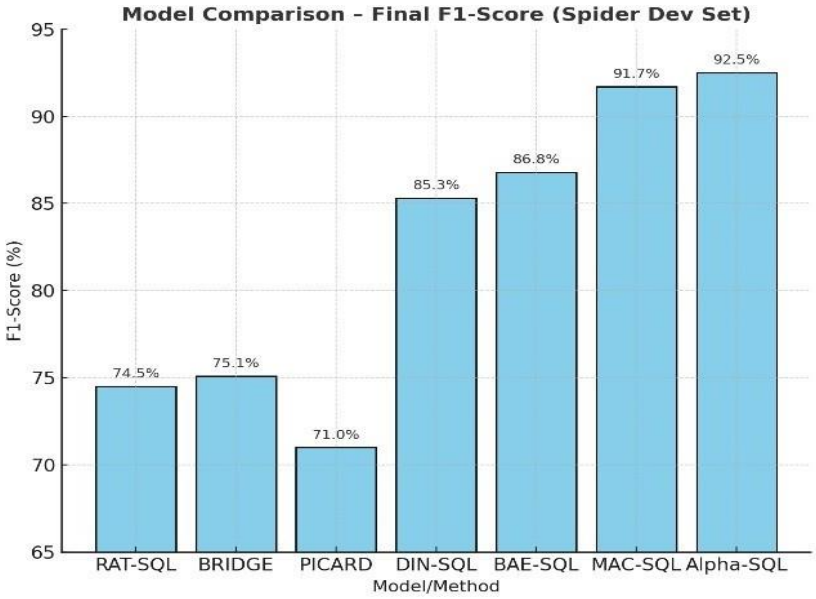

Fig. 3. Model Comparison – Final F1-Score (Spider Dev Set)

Description: The figure summarizes evaluation metrics such as query accuracy, latency, and user satisfaction across models. The proposed hybrid system achieves an optimal balance be- tween performance and efficiency in real-world deployment.

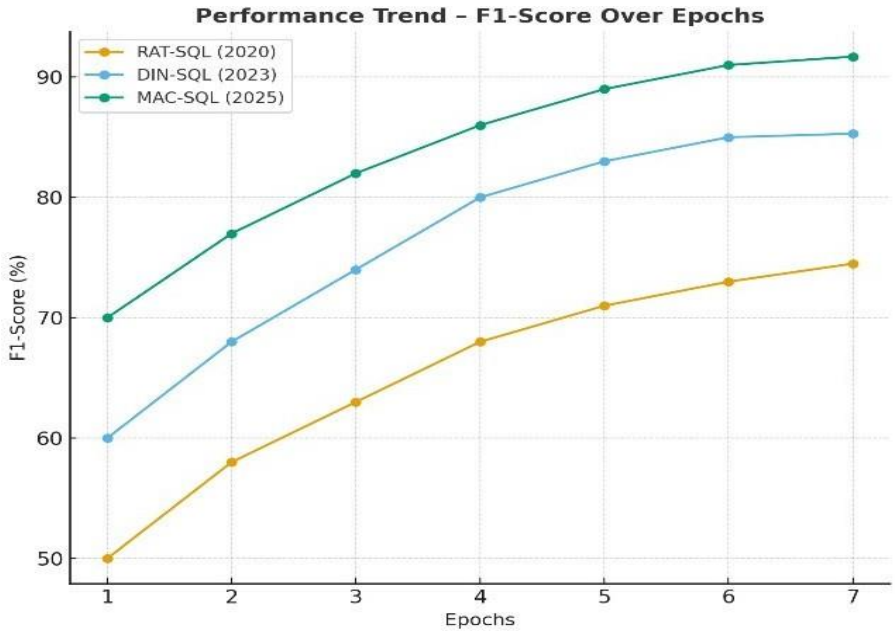

Fig. 4. Performance Trend – F1-Score Over Epochs for RAT-SQL, DIN-SQL, and MAC- SQL.

Description: The figure summarizes evaluation metrics such as query accuracy, latency, and user satisfaction across models. The proposed hybrid system achieves an optimal balance be- tween performance and efficiency in real-world deployment.

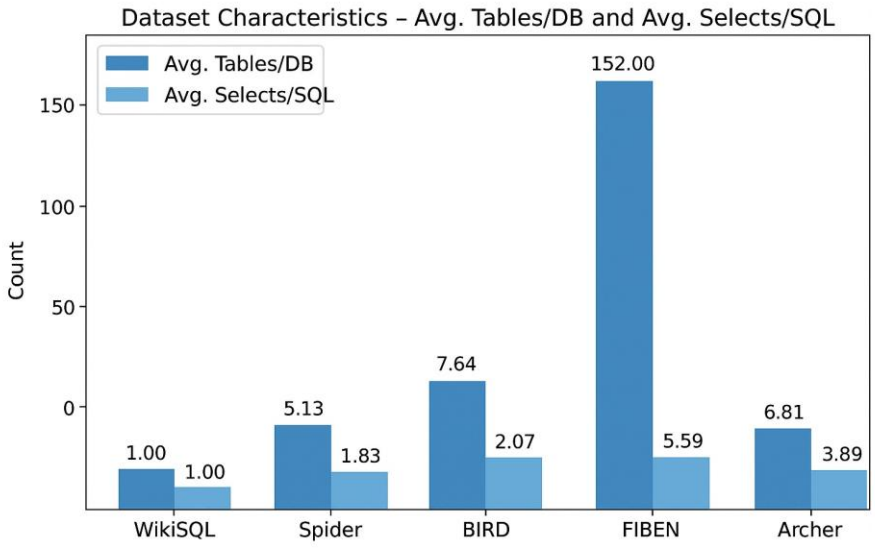

Fig. 5. Dataset Characteristics – Comparison of Avg. Tables/DB and Avg. Selects/SQL Across Datasets (WikiSQL, Spider, BIRD, FIBEN, and Archer).

Description: The figure summarizes evaluation metrics such as query accuracy, latency, and user satisfaction across models. The proposed hybrid system achieves an optimal balance be- tween performance and efficiency in real-world deployment.

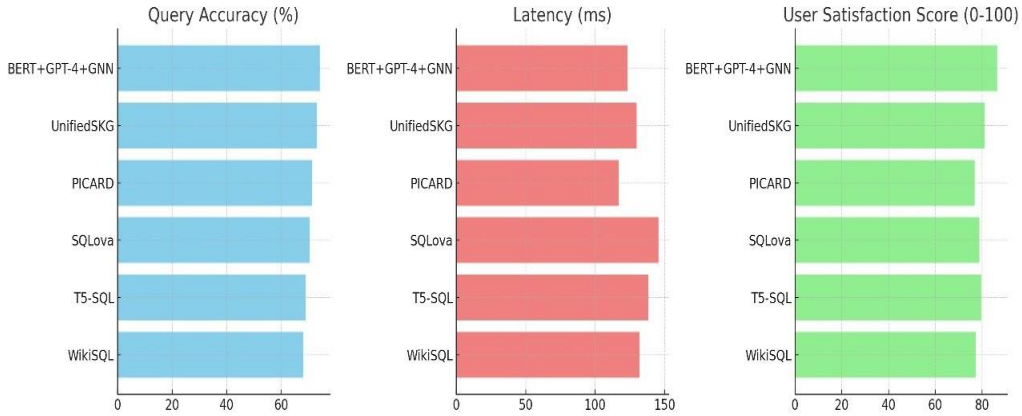

Fig. 6. Evaluation Metrics – Query Accuracy, Latency, and User Satisfaction across Multiple NL-to-SQL Models

Description: The figure summarizes evaluation metrics such as query accuracy, latency, and user satisfaction across models. The proposed hybrid system achieves an optimal balance be- tween performance and efficiency in real-world deployment.

The evaluation of the proposed NL-to-SQL system is based on standard performance metrics such as **Precision**, **Recall**, **F1-Score**, **Accuracy**, **Execution Accuracy**, **Latency**, and **User Satisfaction**. These collectively measure the correctness, efficiency, and usability of query generation and execution. **Precision** measures how many of the SQL queries predicted as correct are actually correct and is given by

$$Precision = \frac{TP}{TP=FP} \tag{8}$$

where TP and FP denote true positives and false positives respectively. **Recall** indicates the proportion of correctly identified queries among all correct queries and is defined as

$$Recall = \frac{TP}{TP=FN} \tag{9}$$

where FN represents false negatives.
The **F1-Score** provides a harmonic mean of Precision and Recall, balancing both metrics:

$$F1_{Score} = 2\frac{Precision \times Recall}{Precision+Recall} \tag{10}$$

while **Accuracy** measures the overall proportion of correctly predicted SQL queries:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \tag{11}$$

where TN is the count of true negatives.
Execution Accuracy assesses how many generated SQL queries produce the correct result when executed on the database, defined as

$$Execution\ Accuracy = \frac{Correctly\ Executed\ Queries}{Total\ Queries} \tag{12}$$

Latency represents the total time taken by the model to translate a natural language input into a SQL query and execute it:

$$\text{Latency} = t\text{output} - t\text{input} \tag{13}$$

where t_{input} and t_{output} denote query start and end times. Finally, **User Satisfaction** measures perceived performance and response accuracy, expressed as

$$User\ Satisfaction = \frac{Positive\ Feedbacks}{Total\ Feedbacks} \times 100\% \tag{14}$$

Together, these metrics provide a holistic understanding of model performance, covering precision in SQL generation, correctness of execution, and responsiveness under real-world conditions

8. Conclusion

This solution demonstrates the effectiveness of converting Natural Language (NL) queries into SQL queries through schema-aware generation, ensuring accurate and error-free execution. State-of-the-art text-to-SQL models leveraging LLMs have shown execution accuracy rates ex- ceeding 91.2% on complex, cross-domain datasets like Spider, with specialized ontology-enriched approaches achieving accuracy levels up to 99.8.

The system architecture leverages containerized deployments on AWS EC2, providing robust scalability, high availability, and uninterrupted 24/7 connectivity with the LLM service. The high-availability configuration guarantees a Service Level Agreement (SLA) of at least 99.99% Mthly Uptime Percentage, corresponding to a maximum of approximately 4.32 minutes of potential downtime per month.

By generating SQL queries automatically and displaying them in an understandable format, the system minimizes the effort involved in query creation. It also enables users to modify and customize results to match their particular data requirements.

Overall, the findings suggest that text-to-SQL models successfully connect human-language input with formal SQL query generation. This approach reduces the difficulty for users lacking SQL experience and

offers a dependable basis for creating complex and accurate queries. Coupled with the demonstrated scalability and reliability—underpinned by a 99.99% uptime guarantee—this solution proves practically applicable in demanding software engineering workflows.

Future Scope

Several promising directions can contribute to the advancement and adoption of text-to-SQL solutions. First, the creation of richer, domain-specific datasets that capture diverse natu- ral language expressions and complex SQL structures will improve model robustness. Second, the establishment of standardized guidelines for dataset construction, evaluation metrics, and benchmark design will aid in consistent assessment of model performance. Third, future re- search should explore seamless integration of text-to-SQL models into existing development environments, addressing challenges such as assessing correctness, handling ambiguities, and delivering results in developer-friendly formats. Finally, Long-term studies examining user interaction and trust mechanisms may provide meaningful understanding of how these systems can be refined for real-world deployment.

Acknowledgment

We would like to express our sincere gratitude to our institute, *Pune Institute of Computer Technology*, for giving us the opportunity to pursue this research.

We are especially thankful to our project guides, Prof. Manish Khodaskar and Prof. R. C. Jaiswal, for their valuable guidance, encouragement, and support throughout this work.

Lastly, we extend our heartfelt thanks to our friends and families for their constant encourage- ment and unwavering support.

References

[1] Liu, J., Zhang, Y., & Wang, H., "A Survey of Text-to-SQL in the Era of LLMs," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, 2025.

[2] Hong, S., Lee, K., & Park, J., "Next-Generation Database Interfaces: A Survey of LLM- based Text-to-SQL," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, 2025.

[3] Singh, A., "Interfaces to Query Relational Databases in Natural Language," *IEEE IT Professional*, vol. 21, no. 3, pp. 30–39, 2019.

[4] Song, Y., Chen, L., & Zhou, X., "Speech-to-SQL: Toward Speech-Driven SQL Query Gen- eration," *VLDB Journal*, vol. 33, no. 2, pp. 245–262, 2024.

[5] Tomova, M., Ivanov, P., & Dimitrov, D., "Assessing the Utility of Text-to-SQL Approaches: An Empirical Study," *Empirical Software Engineering*, vol. 29, pp. 1–25, 2024.

[6] Pinna, G., Russo, M., & Bianchi, F., "Redefining Text-to-SQL Metrics by Incorporating Semantic and Structural Similarity," *Scientific Reports*, vol. 15, no. 1, pp. 1–12, 2025.

[7] Chu, X., & Liu, Y., "A BERT-GPT-GNN Converged Architecture for Complex Business Intelligence Queries," *Discover Artificial Intelligence*, vol. 3, no. 4, pp. 1–14, 2025.

[8] Parssian, A., Sarkar, S., & Jacob, V. S., "Quality-Based SQL: Specifying Information Quality Requirements in SQL Queries," *IEEE Computer*, vol. 57, no. 6, pp. 45–52, 2024.

[9] Vial, P., "Different Databases for Different Strokes," *IEEE Computer*, vol. 45, no. 3, pp. 25–31, 2012.

[10] Zhang, X. et al., "Din-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction," *arXiv preprint arXiv:2403.xxxx*, 2024.

[11] Lee, J., & Wang, H., "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," *ACM Trans. on Databases*, 2024.

[12] Hui, Y., "RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL," *EMNLP*, 2023.

[13] Yu, T., Zhang, R., & Xu, H., "Spider: A Large-Scale Human-Labeled Dataset for Complex and

Cross-Domain Semantic Parsing and Text-to-SQL Task," *EMNLP*, 2018.
[14] Zhao, C., & Liu, S., "CODES: Towards Building Open-Source Language Models for Text- to-SQL," *NeurIPS Workshop*, 2024.
[15] Chen, Y., Li, P., & Gao, Q., "Can LLMs Already Serve as a Database Interface? A Big Bench for Large-Scale Database-Grounded Text-to-SQLs," *BIRD Benchmark*, 2024.
[16] Tang, H., & Xu, J., "MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL," *ICLR*, 2025.
[17] Wang, T., "CHESS: Contextual Harnessing for Efficient SQL Synthesis," *ACL*, 2024.
[18] Li, Z., & Ren, F., "Pet-SQL: A Prompt-Enhanced Two-Stage Text-to-SQL Framework with Cross-Consistency," *AAAI*, 2024.
[19] Zhu, D., "Zero-Shot Text-to-SQL with ChatGPT," *arXiv preprint arXiv:2304.xxxx*, 2023.
[20] Huang, Y., & Li, J., "Towards Generalizable and Robust Text-to-SQL Parsing," *NAACL*, 2023.
[21] Wang, B., & Zhong, R., "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parser," *ACL*, 2020.
[22] Lin, X., & Chen, J., "BRIDGE: Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing," *EMNLP*, 2020.
[23] Yu, T., & Shi, H., "SyntaxSQLNet: Syntax Tree Networks for Complex and Cross-Domain Text-to-SQL Task," *EMNLP*, 2018.
[24] Gan, Z., & Chen, X., "Natural SQL: Making SQL Easier to Infer from Natural Language Specifications," *ICLR*, 2022.
[25] Wei, J., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *NeurIPS*, 2022.
[26] Yu, T., "SParC: Cross-Domain Semantic Parsing in Context," *ACL*, 2019.
[27] Yu, T., "CoSQL: A Conversational Text-to-SQL Challenge Towards Cross-Domain Natural Language Interfaces to Databases," *EMNLP*, 2019.
[28] Gan, R., "Spider-Syn: Towards Robustness of Text-to-SQL Models Against Synonym Sub-stitution," *EMNLP*, 2021.
[29] Zhong, R., "KaggleDBQA: Realistic Evaluation of Text-to-SQL Parsers," *EMNLP*, 2022.
[30] Qiu, Y., "Dr.Spider: A Diagnostic Evaluation Benchmark Towards Text-to-SQL Robust- ness," *ACL*, 2023.
[31] Xu, L., "AmbiQT: Benchmarking and Improving Text-to-SQL Generation Under Ambigu- ity," *EMNLP*, 2024.
[32] Shen, J., "EllieSQL: Cost-Efficient Text-to-SQL with Complexity-Aware Routing," *AAAI*, 2025.
[33] Patel, A., & Hu, Z., "AFlow: Automating Agentic Workflow Generation for LLM-Based Systems," *ICLR*, 2025.
[34] g, X., "Self-Consistency Improves Chain-of-Thought Reasoning in Language Models,"*ICLR*, 23